

Full Audit — Things To Check

A fresh read of the whole web product: the Django backend (report maths, sessions, scheduled jobs, access control, deployment) and the client panel (reports and exports, data fetching, auth, charts). Every item below is something found in the code that looks wrong, risky, or needs a decision. **Nothing here has been reproduced in the running app.** Treat each one as a claim to verify, not a fact. Some will turn out to be intentional and some will not be worth changing — say so and we close them.

7 September 2026 · audited `wergonic-django-backend` dev `f5eee68` and `wergonic-web-apps` dev `674ad24c` (both 6 Sep 2026)

Scope: 286 backend Python files (migrations and tests excluded), 1,853 client-panel TypeScript files, the shared `packages/ui`, and the DigitalOcean app specs.

Not in scope: the admin panel (on hold) and the `/prototype` folder.

P0 = wrong or lost data a client could act on · **P1** = wrong behaviour a user would notice · **P2** = smaller, cleanup, or a decision needed · **Access** = nobody hits it by using the product; it needs a deliberately crafted request

5

P0

21

P1

32

P2

4

ACCESS

7

AREAS

READ THIS FIRST

Three things stand out above everything else.

- Nothing runs cron, in production or in staging.** Each app spec has three components: a migrate job, the web service and a Celery worker. None of them starts `cron`, and `cron-entrypoint.sh` is never used. So the four jobs listed in `CRONJOBS` never fire — including `session_check`, which ends a session that has run past its time limit, and `recover_stuck_assessments`, which unsticks a RAMP assessment left at "in progress" after a worker restart. Both were written to fix real problems.
- Staging runs with DEBUG on.** `DEBUG: 'True'` in the staging app spec. Anyone who triggers an error on `staging.cloud.wergonic.com` gets a full Django error page with the traceback, the settings and the SQL. And because Sentry is only started when DEBUG is off, **no staging error has ever reached Sentry** — even though we set up Sentry alerting for it. Django also keeps every SQL query in memory while DEBUG is on, so the process grows all day.
- A task report can be built from a different task's numbers.** The PDF export waits on `isLoading`, but React Query is configured globally with `keepPreviousData`, so when the task changes `isLoading` stays false and the previous task's data is still what is in hand. The export then fires on a 100 ms timer. Heart rate, temperature, TNO and per-task stats are not waited on at all.

A · Report numbers 10 items

These change the values that end up in front of the client.

A1 A whole limb's angles are sign-flipped when the average leans backward **P0**

`detect_and_correct_inversion` takes the mean of the forward-bending series and, if it is below -5° , negates every value. It runs on head and trunk before the posture bands are counted. A worker who genuinely spends most of a task leaning back has all of that reported as bending forward. There is no log line, no flag on the session, and nothing in the report that says it happened.

`user_sessions/calculations/correct_inversion.py` + `risk_assessment_posture.py`

A2 Missing angles are counted as 0°, and angles are cut off rather than rounded P1

`_to_int` does `to_numeric(errors="coerce").fillna(0).astype(int)`. Two things follow from that one line. Any row whose angle could not be read becomes a perfect neutral 0° and lands in the "0–10°" band instead of being dropped. And `astype(int)` truncates toward zero, so 10.9° counts as 10° and –10.9° counts as –10° — the rounding goes in opposite directions above and below zero, which moves values across band boundaries.

`user_sessions/calculations/risk_assessment_posture.py`

A3 Bar chart percentages are cut to whole numbers, then averaged instead of added P1

`calculate_bar_data` works out each angle's share of the time, casts it with `int()` (0.9 % becomes 0), then casts the angle itself to an integer — which merges 10.2° and 10.7° into one label. The two shares for that merged label are then *averaged*, not summed. The bars under-report and the chart does not add up to 100 %.

`calculate_bar_speed_of_mouvement` has the identical code.

`user_sessions/calculations/stats.py`

A4 The "Total" row of the task table is measured over different data than the rows above it P1

Each task row is built from readings inside the task window and inside the session's own start and end. The Total row calls `calculate_percentiles`, which is raw SQL filtered on `measurement_id` alone — no session bounds, no task windows. So Total can include readings from before the session started or after it ended, and it is not the total of what is listed above it. Its duration column, meanwhile, *is* the sum of the task durations.

`user_sessions/calculations/stats.py` – `calculate_tasks_percentiles`

A5 The same session percentiles are calculated twice, two different ways P1

`calculate_percentiles` uses Postgres `percentile_cont` with no session bounds.

`calculate_session_total_percentiles` pulls every row into Python and uses `numpy.percentile`, bounded by the session. They are stored in different fields and can disagree for the same session and the same limb. The numpy one also loads the whole measurement into memory — millions of rows for a long recording.

`user_sessions/calculations/stats.py`

A6 Wrist velocity percentiles read a different column than the wrist velocity charts P1

The bar and line charts switch to `ref_gyro_angular_velocity` when the limb is the hand. The percentile calculations always read `angular_velocity`. So the wrist's median and 90th-percentile velocity come from one signal while the chart beside them is drawn from another.

`user_sessions/calculations/stats.py`

A7 Hand-distance zone times are scaled up to the full session length P1

`calculate_wrist_distance_line_data` measures green / yellow / red time from the samples it has, converts to percentages, then multiplies by the whole session duration. If the forearm sensor only recorded 20 minutes of an 8-hour session, the report still shows zone times adding up to 8 hours. Confirm whether that projection is the intended presentation.

`user_sessions/calculations/stats.py`

A8 Negative velocities fall outside every bin but stay in the denominator P2

Movement bins start at 0 and the denominator is the total time of all non-null velocity rows. Any negative angular velocity counts toward the denominator but into no bin, so the bars quietly sum to less than 100 %. Check first whether negative values can occur at all — if they cannot, this is nothing.

`user_sessions/calculations/risk_assessment_posture.py` – `_vel_rows`

A9 Head at exactly –5° is counted in two places P2

The distribution puts –5° in the "–5–0°" band (≥ -5), while the risk figure "Head bending backward $\leq -5^\circ$ " includes it (≤ -5). Angles are stored as whole degrees, so –5 is a value that really occurs. Small, but the two numbers on the same page disagree by whatever time sits at exactly that angle.

`user_sessions/calculations/risk_assessment_posture.py`

A10 The RAMP 0–7 score averages two unrelated percentages P2

For questions 1.1 and 1.3 the score is `int((mean of the two percentages / 100) * 7)` — the forward-bending share and the side-bending share averaged together, then truncated. That is a different number from the RAMP verdict itself, which comes from `calculate_postures_1`. Confirm what this score is used for and whether averaging is what it should do.

`ramp/calculations/ramp_calculator.py`

B · Scheduled jobs & session data 11 items

Backend work that runs on its own, and the places where session data can be lost or mangled.

B1 No cron component exists, so four scheduled jobs never run P0

The production and staging app specs each contain a migrate job, the web service and a Celery worker — nothing that runs `cron`, and nothing that uses `cron-entrypoint.sh`. The `CRONJOBS` block in settings is therefore dead configuration. The two that matter: `session_check` (ends a session that has passed its `max_duration_minutes`) and `recover_stuck_assessments` (marks a RAMP assessment that has been "in progress" for over an hour as failed so it can be retried). Check what actually happens today to a session nobody stopped, and to an assessment whose worker was killed mid-run.

`.do/app.production.yaml`, `.do/app.staging.yaml`, `core/settings.py` — `CRONJOBS`

B2 The in-process scheduler starts once per web instance, and production scales to three P1

`SchedulerConfig.ready()` calls `start_scheduler()`, so every web process gets its own `APScheduler`. The production service is set to `min_instance_count: 1`, `max_instance_count: 3`. Under load, `archive_sessions`, `session_status_check` and the Sentry poller all run up to three times at once. The `threading.Lock` guards only protect one process, so they do not help here. Duplicate Google Chat alerts are the visible symptom; the archive job is the one worth checking properly.

`scheduler/apps.py` + `scheduler/jobs.py`

B3 Two scheduling systems are configured for overlapping jobs P2

`CRONJOBS` (`django-crontab`) and `start_scheduler` (`APScheduler`) both schedule `session_status_check` and `archive_sessions`, at different intervals — every minute versus every five hours, and midnight in both. Only `APScheduler` currently runs (see B1). Pick one system and delete the other, so what is scheduled matches what happens.

`core/settings.py` + `scheduler/jobs.py`

B4 An Excel import in "override" mode deletes the organisation's entire work-cycle tree P0

`safe_delete_all_entities` is `Factory.objects.filter(organization=...).delete()`. Every foreign key down the chain cascades, so it removes all lines, workstations, operators, work cycles and tasks — and because `WorkCycleEvent.task` also cascades, **the work-cycle events of past recorded sessions go with them**. Old sessions lose their task timeline permanently. Confirm what "override" is meant to replace, and whether the UI warns about it.

`workcycles/views.py` — `ImportWorkCyclesExcelView.safe_delete_all_entities`

B5 The two work-cycle importers read the time column in different units P1

Task duration is stored in whole seconds. The Excel importer does `float(row["Time"]) * 60`, so it treats the column as minutes. The Avix JSON importer does `int(round(float(time)))`, so it treats the same idea as seconds. One of the two produces durations 60× off. Check what unit Avix actually exports.

`workcycles/views.py` — `ImportWorkCyclesView` vs `ImportWorkCyclesExcelView`

B6 Editing part of a work-cycle event returns a 500 P1

`WorkCycleEventSerializer.validate` reads `data['start_time']` and `data['end_time']` directly. On a PATCH that sends only one of them, the other key is not in the validated data and the lookup raises `KeyError` — which surfaces as a 500, not a validation error.

`workcycles/serializers.py`

B7 Removing a team member can delete the account and their worker profiles outright P1

The bulk delete strips the membership, then — if that was the person's only membership — deletes every `WorkerProfile` they own and the user record itself, including from Firebase. Sessions survive (both foreign keys are `SET_NULL`), but they come back with no worker and no recorder attached. Check whether "remove from team" is supposed to be a permanent account deletion, or whether it should just end the membership.

`organizations/views/org_user/list_create_delete.py` — `delete()`

B8 Searching the team list by name may return a 500 P1

`search_fields` starts with `"=id"`, which DRF turns into `id__iexact`. Django prepares that lookup by converting the search term to an integer, and a name is not one. Try searching the team page for a person's name and see what comes back; if it errors, dropping `"=id"` (or moving it behind a numeric check) is the fix.

`organizations/views/org_user/list_create_delete.py` – `OrganizationUserView`

B9 Merging sessions picks an assessment type at random when they disagree P2

`merged_assessment_type = list(assessment_types)[0]` takes an arbitrary element out of a Python set. Merge the same two sessions twice and you can get a different answer. Decide the rule: first session wins, or refuse the merge when they differ.

`user_sessions/views/session_views.py` – `MergeSessions.merge_sessions`

B10 The import success message counts rows attempted, not tasks created P2

`bulk_create(..., ignore_conflicts=True)` returns the objects it was handed, including the ones the database skipped. So "imported successfully (N tasks processed)" reports the file's row count, and a re-import of a file that changes nothing still says it processed everything.

`workcycles/views.py` – `ImportWorkCyclesExcelView`

B11 Every authenticated request writes a row to the users table P2

`FirebaseAuthentication` updates `last_seen` on every single API call. Loading one dashboard page is a dozen writes to the same row. It also uses `datetime.now()` rather than `timezone.now()`, which is a naive timestamp in a project that has time zones enabled. Consider only writing it when the stored value is more than a few minutes old.

`firebase_auth/authentication.py`

C · Access & organisation scoping 8 items

None of these are reached by using the product normally — our own apps never send the requests involved. They are here so we can decide priority, not because a customer is about to hit them.

C1 Work-cycle writes accept a parent from any organisation ACCESS

Reading factories, lines, workstations, operators, work cycles and tasks is scoped to the caller's organisation. Writing is not: the serializers accept the parent id from an unrestricted queryset (`Operator.objects.all()` , `TaskModel.objects.all()` , and plain model fields for the rest). So a crafted request can hang a new line under another company's factory, or move one of ours into their tree. `FactorySerializer` also exposes `organization` as a writable field on update.

`workcycles/serializers.py`

C2 The websocket endpoint can change any session by id ACCESS

`SessionStatusConsumer.status_message` takes an id off the socket message, loads that session and writes its status and device list — with no check that it belongs to the caller's organisation. It also keys the room off `user.organization` , the legacy single field, rather than the active organisation. The middleware in front of it is commented "not used, replaced by firebase firestore", but it is still wired up in `core/asgi.py` . If nothing uses it any more, deleting the whole websocket path is the cleanest answer.

`user_sessions/consumers.py` + `core/asgi.py`

C3 Bulk team delete accepts any user id ACCESS

The endpoint checks that the *caller* is an admin, then loops over whatever ids were sent without checking the targets belong to the organisation. Removing the membership is harmless in that case (there is none), but the next step deletes any user who ends up with no memberships at all — which includes another organisation's pending invitee or someone mid-way through referral registration.

`organizations/views/org_user/list_create_delete.py` – `delete()`

C4 Role checks ask "in any organisation", not "in this one" P1

`LevelOnePermission` and `LevelTwoPermission` pass if the user holds the role in *any* live membership. Someone who is an org admin at company A and an ergonomist at company B satisfies the admin check while working in B. Most views narrow it again with an organisation-scoped queryset, so the practical effect is limited — but the permission itself is not saying what it reads as. The client panel has the mirror of this: see E6.

`users/permissions/level_1.py`, `level_2.py`

C5 Any signed-in user can list every organisation on the platform ACCESS

`ListAllOrganizationNames` is `IsAuthenticated` and returns the names of every active organisation. Any customer account can read our full client list. Check what the endpoint is for — if it exists for a sign-up or request-access flow, it probably belongs behind a search term with a minimum length, or on the unauthenticated side with a rate limit.

`organizations/views/organizationView.py`

C6 The API documentation and the OpenAPI schema are public P2

`/api/v1/docs/` and `/api/v1/openapi.json` are served with `AllowAny` and `public=True` , in every environment. Anyone can browse the full surface of the API. Confirm whether that is deliberate; if not, the usual answer is to serve them on staging only.

`core/urls.py`

C7 A session can be recorded against any user id P2

`StartSession.post` does `User.objects.get(id=ergonomist_id)` with no organisation check, so the "recorded by" field can be set to somebody outside the organisation. The worker is checked properly two lines later. If that id does not exist the exception is swallowed by the outer handler and comes back as a 500 rather than a 400.

`user_sessions/views/session_views.py`

C8 Every view is public by default P2

`DEFAULT_PERMISSION_CLASSES` is `AllowAny`, so a view is only protected if it sets `permission_classes` itself. Today they all do except the sign-in-link and register endpoints, which is correct — but the default means the next view someone adds is open unless they remember. Flipping the default to `IsAuthenticated` and marking the three public views explicitly removes the whole class of mistake.

core/settings.py — REST_FRAMEWORK

D · Client panel — reports & exports 8 items

The PDF and Excel exports a client actually receives.

D1 A task report can contain the previous task's numbers P0

The export waits on `isReducedStatsLoading`. React Query is configured globally with `keepPreviousData: true`, so on a task switch that flag stays false while the old data is still being served — `isFetching` is what turns true. The export then runs on a `setTimeout(..., 100)`. Pick a task, generate its report, and check the numbers against the on-screen values for that task.

`features/sessions/components/SingleSessionReportExportButton/ReportExportButton/ReportExportButton.tsx`

D2 Only the posture query is waited on; four others are not P1

Same export. Heart rate, temperature, TNO and per-task stats are read straight out of `useTabData2` with no loading check at all. On a slow connection the PDF can be built before any of them have arrived, and those sections come out empty with no error.

`ReportExportButton.tsx` + `features/sessions/hooks/useTabData.tsx`

D3 The heart-rate page and the assessment summary never appear in the session PDF P1

The template gates its pages on `includedSections["hr_stats"]` and `includedSections["assessment_summary"]`. The object the export builds contains neither key, so both are always false. It does contain `risk_level_indicators` and `intervention_suggestions`, which nothing reads. Check what the exported PDF is supposed to contain and line the two lists up.

`ReportExportButton.tsx` + `features/sessions/components/ResultsPdfTemplate/ResultsPdfTemplate.tsx`

D4 The assessment summary page has no data source, and its labels match nothing P1

`AssessmentResultPage` renders from a `groupedRAMPData` prop that no caller passes. The only function that could build it, `transformSessionStatsToGroupedAssessmentData`, is exported but never called anywhere in the repo. And if it were called, its three velocity groups would come out empty: it filters for labels `"> 60°/s"`, `"30°/s - 60°/s"`, `"< 30°/s"`, while the backend sends `"> 30°/s"`, `"15°/s - 30°/s"`, `"0°/s - 15°/s"` for arms and a different set again for the wrist. Decide whether this page is wanted; if it is, the labels need fixing too.

`features/sessions/utils/PdfAssessmentFormatter.ts`

D5 The exported PDF has no file extension and ignores the app's own naming rules P2

The download name is built inline as `measurement_<id>_<datetime>_<worker>`, with no `.pdf` on the end. `src/utils/fileNames.ts` exists precisely to keep export names consistent with the backend's, and this export does not use it. Download one and see what the file is called on disk.

`ReportExportButton.tsx` vs `utils/fileNames.ts`

D6 The comparison PDF is rendered on page load, not on click P1

`DownloadReportsButton` calls `usePDF` during render, so opening the multi-session comparison page builds the whole PDF whether or not anyone wants it — and rebuilds it every time the data changes. It also passes `data!`, which is undefined on the first render.

`pages/dashboard/MultiSessionDetailsPage/DownloadReportsButton/DownloadReportsButton.tsx`

D7 Two report exports, one honours the saved template and one ignores it P2

The "generate report" flow lets a user pick a saved report template and reads it back before building the report. The PDF/Excel export button next to it hardcodes every section to `true`. Whichever behaviour is right, having both is confusing — a user who unticks sections in their template still gets them in the PDF.

`ReportExportButton.tsx` vs `FinishedSessionSettingsButton/GenerateReportButton`

D8 The comparison filename is stamped twice P2

`comparisonName` and `comparisonPdf` are each given their own `new Date()`. Only the date is used, so the two can only differ if the page is open across midnight. Listed for completeness — probably not worth changing.

`DownloadReportsButton.tsx`

E · Client panel — behaviour 7 items

E1 Any 401 signs the user out P1

The response interceptor calls `forceSignOut` on every 401, whatever the reason. The backend answers 401 for a deactivated membership (which is what this was built for) but also for an expired or malformed token and for "authentication service unavailable" — which is what it returns when Firebase itself is having a bad minute. In that case every signed-in user is thrown out to the login page. Check whether the sign-out should be limited to the `membership_revoked` code.

`services/axios.ts` + `services/forcedSignOut.ts`

E2 The error boundary resets itself while rendering P1

`ErrorFallback` calls `resetErrorBoundary()` in its own render body and then redirects to the error page. Resetting means the children are retried, so an error that happens every time will throw, reset, throw again. Worth reproducing with a component that always fails and seeing whether the page loops or settles.

`providers/GlobalAppProvider.tsx`

E3 Line charts average their buckets, so short peaks never show P1

`downsample` splits the series into as many buckets as there are target points and plots the mean of each. A brief extreme posture inside a bucket is averaged away and never reaches the screen. It also takes the first timestamp of the bucket as the x value, so points drift left. If the chart is meant to show worst cases, min/max-preserving downsampling is the usual answer.

`utils/downsampling/algorithm.ts`

E4 The notifications listener subscribes before it knows who the user is P2

`useSubscribeToNotifications` runs its effect with no guard, so on first render it listens on `organizations/undefined/users/undefined/notifications`. There is also no error callback on `onSnapshot`, so a permission failure goes nowhere. Add the guard and an error handler.

`features/dashboard/.../NotificationsButton/NotificationsButton.hooks.ts`

E5 Errors are only ever a three-second snackbar P2

`autoHideDuration={3000}` for everything, success and failure alike, with no error state left on the page afterwards. A failed save, a failed export and a failed load all vanish in three seconds with nothing to go back to. Consider keeping error snackbars until dismissed.

`providers/NotistackProvider.tsx`

E6 Write permission falls back to the account's global role P2

`useVerifyWritePermission` reads `consulting_role`, and when that is missing falls back to `role` — which is the account-level field, not the role in the organisation being viewed. The team list deliberately does the opposite and shows a dash rather than another client's role. Worth making the two consistent. Separately: this hook only grants write to admins, while the backend lets ergonomists write — confirm which is intended.

`hooks/permissions.ts`

E7 Session status is written from the socket without checking the organisation P2

Client-panel side of C2. Nothing in the client panel opens that websocket any more — live sessions come from Firestore. If that is settled, the socket route, the consumer and the middleware can all go.

no client-panel caller found; server side in `user_sessions/consumers.py`

F · Configuration & deployment 8 items

F1 Staging runs with DEBUG on, so it reports nothing to Sentry P0

DEBUG: 'True' in the staging app spec. Three consequences. Django serves full error pages with traceback, settings and SQL to anyone who triggers a 500 on the public staging host. Sentry is initialised inside `if not DEBUG`, so staging errors have never been reported — despite `environment = "staging"` being set up for exactly that. And Django accumulates every SQL query in memory for the life of the process.

`.do/app.staging.yaml + core/settings.py`

F2 None of Django's HTTPS and cookie settings are set P1

No `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE` or HSTS anywhere in settings, while `/api/v1/admin/` is served publicly and uses session cookies. `SECURE_PROXY_SSL_HEADER` is set, so Django knows the request came in over TLS — the cookies just are not marked secure.

`core/settings.py`

F3 ALLOWED_HOSTS = ["*"], and staging allows every CORS origin P2

Hosts are unrestricted in every environment. On anything that is not PROD, `CORS_ALLOW_ALL_ORIGINS = True` — so any website can make browser calls to the staging API with the caller's credentials. Staging holds real-shaped data.

`core/settings.py`

F4 Throttling is set high enough to have no effect P2

1,000 requests per minute for anonymous callers and 2,000 for authenticated ones. The public endpoints that matter — password reset, sign-in link, invitation lookup by token — are all covered by the anonymous rate. The `custom_post` scope of 10/minute exists but check where it is actually applied.

`core/settings.py`

F5 A local checkout points at the staging database by default P1

The database block is "PROD or staging" — anything that is not `ENVIRONMENT=PROD` uses the `STAGING_DATABASE_*` credentials. So running the backend locally connects to the shared staging database, and a stray migration or management command lands there. A local Postgres should be the default and staging should be opt-in.

`core/settings.py`

F6 The health endpoint returns the raw database error P2

On failure it responds with `{"detail": str(exc)}` to an unauthenticated caller. A psycopg error string can carry the host, the database name and the user. Log the detail, return only the status.

`core/health.py`

F7 The Dockerfile does not run entrypoint.sh P2

`entrypoint.sh` starts Daphne with `--proxy-headers` and tuned ping timeouts; the Dockerfile's `CMD` starts a plain Daphne with none of that. Confirm which one DigitalOcean actually runs, then delete the one that does not. Related to B1: `cron-entrypoint.sh` is not run by anything either.

`Dockerfile, entrypoint.sh, cron-entrypoint.sh`

F8 Channels is configured with an in-memory layer P2

`InMemoryChannelLayer` is per-process, so nothing published on one web instance reaches a socket held by another. With autoscaling up to three, the websocket path cannot work reliably. If C2/E7 end in deleting the websocket, this goes with it — along with the `channels` dependency.

`core/settings.py + core/asgi.py`

G · Cleanup & dead code 10 items

G1 `pie_stats_swapped.py` — 661 lines, deprecated, imported nowhere P2

A near-copy of `pie_stats.py` with its own header saying it was merged into the other one and is kept "for backwards compatibility". Nothing imports it. It also rounds differently from the file that replaced it, so anyone who does import it by accident gets different numbers. Delete it.

`user_sessions/calculations/pie_stats_swapped.py`

G2 `useFirestoreListener` is unused — and leaks a listener when it errors P2

No component calls it. If it were used, its error path schedules `setTimeout(subscribe, 3000)` with no way to cancel, so a component unmounted between the error and the retry ends up with a Firestore listener nobody can unsubscribe, retrying forever. Delete the file.

`hooks/useFirestoreListener.ts`

G3 `React Query Devtools` is still rendered and still a runtime dependency P2

`<ReactQueryDevtools />` is rendered unconditionally in the global provider, and `@tanstack/react-query-devtools` sits in `dependencies` rather than `devDependencies`. The component no-ops in a production build, but the package is still shipped. This one has been on the list for a while.

`providers/GlobalAppProvider.tsx` + `package.json`

G4 Two declared dependencies are never imported P2

`file-saver` and `html-react-parser` appear in the client panel's dependencies with no import anywhere in `apps/client-panel` or `packages/ui`. `@types/file-saver` can go with them.

`apps/client-panel/package.json`

G5 `logging.basicConfig` at import time overrides the project's logging config P2

`stats.py` calls `logging.basicConfig(level=INFO, format=...)` when it is imported, which reconfigures the root logger for the whole process and undoes the structured setup in settings. It also uses the bare `logging.info` rather than a named logger.

`user_sessions/calculations/stats.py`

G6 `Matplotlib` and a debug plotter live in the MEC calculation path P2

`mec_mapping.py` imports `matplotlib.pyplot` at module level and carries `plot_peaks_and_static_periods`, which writes a PNG. Pyplot keeps global figure state and is not safe to call from more than one thread. If the plots are still useful for debugging, put the import inside the function; if not, delete both.

`user_sessions/utils/mec_mapping.py`

G7 The websocket middleware is marked unused but is still wired up P2

First line of the file: "not used, replaced by firebase firestore in real time tasks". It is still installed in `core/asgi.py`. It swallows every failure with a bare `except:` and then returns `None`, which is not a valid ASGI response. Same decision as C2 and E7.

`user_sessions/middleware.py`

G8 Debug logging left in a scheduled command P2

`session_check` does a bare `print(session.id)` for every candidate session on every run. Not currently a problem only because the command never runs (B1).

`user_sessions/management/commands/session_check.py`

G9 Timing hacks holding the report export together P2

Three `setTimeout` calls (50 ms, 50 ms, 100 ms) used to sequence state updates before the PDF is generated, plus `/* eslint-disable react-hooks/rules-of-hooks */` at the top of the PDF template because hooks are called conditionally inside it. This is the same code as D1 and D2 — worth restructuring once rather than patching three times.

`ReportExportButton.tsx` + `MultiSessionPDFTemplate.tsx`

G10 142 as any casts outside the prototype P2

Counted across the client panel with the prototype folder excluded. Several are on API responses, where the cast is what lets a shape mismatch through to the screen instead of failing the build — D4's label mismatch is that pattern. Not a task on its own; worth agreeing that new code does not add to it.

apps/client-panel/src

CHECKED AND FINE

Things that looked wrong on first read and are not.

Recorded here so nobody re-reports them. **Snackbar severity** : notistack normally takes `variant` , but the app registers a custom `NotistackAlert` that declares `severity` as a variant override — correct as written. **Active-session statuses**: the Firestore query mixes enum members and bare strings, which reads like duplicates; the enum maps `ONGOING` to `"RECORDING"` and `INTERRUPTED` to `"DISCONNECTED"` , so the list is five distinct values as intended. **Assessment PDF posture labels**: these do match what the backend sends — only the velocity labels do not (D4). **Logo upload on organisation update**: mutating `request.data` looks like it would fail on an immutable `QueryDict`, but DRF hands back a mutable copy whenever files are present. **Session deletion and team deletion**: `recorded_by` and `worker` are both `SET_NULL` , so removing a person does not delete their sessions (B7 is about the profile and the account, not the sessions).

Wergonic web app audit — 7 September 2026 · `wergonic-django-backend` `f5eee68` · `wergonic-web-apps` `674ad24c`
Static code read only. No item here has been reproduced in the running app. Please verify each one and push back on anything that is intentional or not worth the change.